D — Self-training, expert iteration, preference learning and parameter-efficient fine-tuning (weekly LoRA pockets)

0. Role in the system

This section informs the **weekly LoRA job**: a batch process that takes the week's verified trajectories (prompt + reasoning trace + actions + verifier verdict), verified-vs-failed pairs and human-corrected pairs, trains a LoRA adapter on the 9B worker with QLoRA inside 12 GB of VRAM, scores the adapter on a held-out set, archives it, and activates the best one. It also covers distillation from the frozen ~35B-A3B senior into the worker, synthetic task generation when weekly data is thin, LLM-as-a-judge rewards for tasks with no executable verifier, and the "one LoRA per domain + replay buffer" policy against forgetting.

After reading this pack an agent must be able to implement: (1) the filtered-SFT loop (STaR / ReST-EM) including rationalization with hints and train-from-base-every-iteration; (2) the DPO loss with LoRA and a frozen reference; (3) the QLoRA memory arithmetic and hyperparameter defaults; (4) the soft-target distillation loss and the sequence-level distillation alternative; (5) the Self-Rewarding judge prompt and the iterative DPO loop; (6) the Absolute Zero task proposer/solver with learnability reward and executor validation; (7) the evaluation and gating rules that decide whether a new adapter replaces the old one.

1. Papers

1.1 STaR: Bootstrapping Reasoning With Reasoning (Zelikman et al., 2022) — arXiv:2203.14465

Problem and claim. Step-by-step rationales improve LM reasoning, but hand-built rationale datasets are expensive and few-shot rationale prompting usually trails direct answer fine-tuning. STaR ("Self-Taught Reasoner") bootstraps the model's own rationale generation: generate rationales with few-shot prompts, keep those whose final answer is correct, fine-tune on them, repeat. For problems the model gets wrong, **rationalization** supplies the correct answer as a hint so the model can produce a rationale that reaches it; that rationale is then trained on *with the hint removed*, as if the model had reasoned to it unaided.

Headline results. Base model GPT-J 6B. - CommonsenseQA dev accuracy: STaR with rationalization 72.5%; STaR without rationalization 68.8%; GPT-J fine-tuned to answer directly 60.0%; few-shot CoT GPT-J 36.6%; few-shot CoT LaMDA 137B 55.6%; GPT-3 fine-tuned direct (about 30x larger) 73.0%. - GSM8K test accuracy: STaR with rationalization 10.7%; without 10.1%; GPT-J direct fine-tune 5.8%; few-shot CoT GPT-J 3.1%. - Arithmetic (multi-digit addition): STaR with rationalization 89.5% overall after 16 iterations vs 76.3% for a baseline trained on 10,000 examples without rationales for 5,000 steps. With rationalization 2-digit accuracy went from under 1% to 32% after one iteration. - Few-shot prompts included during fine-tuning raised CQA from 60.9% to 68.8% (no rationalization) and 69.9% to 72.5% (with rationalization).

Method — equations. (Section 3.1) Latent-variable view with rationale r:

$$p_M\left(\, y \mid x \,\right) \;=\; \sum_{r} p\left(\, r \mid x \,\right)\; p\left(\, y \mid x, r \,\right)$$

x problem, y answer, r rationale, M the model. Expected reward over the dataset ($\mathbb{1}$ indicator, $\hat{r}_i, \hat{y}_i$ sampled rationale and answer, y_i ground truth):

$$J\left(\, M, X, Y \,\right) \;=\; \sum_{i} \mathbb{E}_{\hat{r}_i, \hat{y}_i \sim p_M\left(\, \cdot \mid x_i \,\right)}\; \mathbb{1}\left(\, \hat{y}_i = y_i \,\right)$$

Policy gradient via the log-derivative trick:

$$\nabla J\left(\, M, X, Y \,\right) \;=\; \sum_{i} \mathbb{E}_{\hat{r}_i, \hat{y}_i \sim p_M\left(\, \cdot \mid x_i \,\right)} \left[\mathbb{1}\left(\, \hat{y}_i = y_i \,\right) \cdot \nabla \log p_M\left(\, \hat{y}_i, \hat{r}_i \mid x_i \,\right) \right]$$

The indicator is what the filtering step implements: gradients from incorrect rationales are dropped, so STaR is an approximation of this policy gradient with greedy sampling and a filtered SFT update.

Method — algorithm. (Algorithm 1, transcribed from the fetched text)

```
Input: pretrained LLM M; dataset D = {(x_i, y_i)}_{i=1..D}; few-shot prompts P
M_0 <- M
for n in 1..N:
    # rationale generation
    for each x_i:  (r_hat_i, y_hat_i) <- M_{n-1}(P, x_i)
    # rationalization: provide the correct answer y_i as a hint
    for each x_i:  (r_hat_i^rat, y_hat_i^rat) <- M_{n-1}(P_hint, x_i, hint = y_i)
    # filter
    D_n     <- { (x_i, r_hat_i, y_i)      : y_hat_i == y_i }
    D_n^rat <- { (x_i, r_hat_i^rat, y_i)  : y_hat_i != y_i  and  y_hat_i^rat == y_i }   # hint stripped from the
stored example
    # fine-tune from the ORIGINAL model M, not from M_{n-1}
    M_n <- finetune(M, D_n ∪ D_n^rat)
```

Steps “rationalization” and the second filter are the rationalization parts; dropping them gives plain STaR. Section 3.1: “we train from the original pre-trained model M instead of continually training one model” — the stated reason is to avoid overfitting. Loop termination: “We run STaR until we see performance saturate, and we report the best results”; “until the performance plateaus”. No data-exhaustion criterion is given; the GSM8K run was stopped by a compute cap at iteration 30 (7,912 steps).

Hint formats: arithmetic — the correct answer is placed after “Target:” and the model writes the scratchpad then restates the answer; CQA — the correct choice is marked “(CORRECT)” in the few-shot exemplar; the hint is omitted when the rationale is added to the training set. GSM8K hint format (not in fetched text — verify in paper Appendix).

Hyperparameters and setup. - GPT-J 6B (28 layers, 16 heads), trained on a single TPU-v3 node. - Fine-tuning schedule: 40 steps in the first outer loop, then +20% steps per outer loop. Arithmetic with rationalization: 300 starting steps, +20 per iteration (starting at 300 without rationalization overfits 1-digit addition). - Optimizer Adam; LR searched in 1e-7..1e-4, **1e-6 best** consistently; 100 warmup steps then constant; batch 8 sequences of length 1024 with packing; no weight decay. - Decoding: greedy / low temperature; temperatures 0.5 and 0.7 performed worse (“can produce correct answers from incorrect reasoning”). - Data: arithmetic 50,000 generated, 10,000 sampled per iteration; CQA 9,741 train; GSM8K 7,473 train / 1,319 test. Few-shot: 10 exemplars per digit length for arithmetic; the 10 CQA prompts of Wei et al. lightly edited. - Iterations: arithmetic 16; GSM8K 36 without rationalization + 10 with, capped at 30th iteration in the main run; CQA until saturation (count not in fetched text).

Ablations that matter. Rationalization: CQA 68.8 to 72.5; GSM8K 10.1 to 10.7; arithmetic learns far faster. Few-shot prompts kept in later iterations: no substantial effect on final performance, but leaving them out makes rationalization gradually worse over long training. Digit curriculum (adding 9- and 10-digit examples at iteration 20): better in-distribution and some OOD success, less stable.

Code / data / license. Code: https://github.com/ezelikman/STaR (found by search; not in fetched paper text). GPT-J via https://github.com/kingoflolz/mesh-transformer-jax. License not stated.

Stated limitations. Few-shot performance “must be above chance” for the first iteration (GPT-2 could not bootstrap even on arithmetic). High-chance tasks (binary) yield many poor rationales; filtering bad reasoning there is open. Without rationalization “improvement ends when the model fails to solve new problems in the training set”. Rationales may not be faithful (answer picked first, then rationalized). If the hint is shown during training the model learns to pick the hinted answer with unsound rationales. Amplifies dataset biases. Limited hyperparameter search.

Use in our design. The weekly job is STaR with the verifier as the correctness check: keep worker trajectories whose tests pass; for tasks the worker failed but the senior or the human solved, run *rationalization* — regenerate the worker’s reasoning trace conditioned on the known-good patch/answer as a hint, keep it only if it reproduces the verified result, and store it with the hint stripped. Train the adapter from the base weights each week (not from last week’s adapter), and schedule steps to grow with the data (STaR’s +20%/iteration is a usable default). Use greedy or low temperature for the data-generation pass; do not raise temperature to get more positives.

PDF: https://arxiv.org/pdf/2203.14465 — Code: https://github.com/ezelikman/STaR

1.2 Beyond Human Data: Scaling Self-Training for Problem-Solving with Language Models (ReST-EM) (Singh et al., 2023; TMLR 04/2024) — arXiv:2312.06585

Problem and claim. Human-written fine-tuning data is limited in quantity and quality, especially for expert problem solving. When a task has a binary or scalar automatic reward (does the answer check out, do the tests pass), the model can improve on its own filtered outputs. ReST-EM alternates Generate (sample from the policy, score with a binary reward) and Improve (fine-tune the **base** model on the kept samples), framed as expectation-maximization for RL. Claim: “self-training with feedback can reduce dependence on human-generated data”; ReST-EM “significantly surpasses fine-tuning only on human data” and “scales favorably with model size”.

Headline results. PaLM 2 models (S, S*, L) accessed via Google Cloud APIs. - MATH test accuracy gain over SFT-on-human-data: +5.94 points (PaLM 2-S), +6.34 (PaLM 2-L). PaLM 2-L: 41% after iteration 2, 41.9% after iteration 3; a single iteration with 3x the samples reaches only 40.3%. - APPS (Introductory) pass@1 gain: +5.6 (PaLM 2-S), +6.4 (PaLM 2-L). - Majority voting with 64 samples on MATH, PaLM 2-L: 48.82 (ReST-EM) vs 44.02 (base). - Pass@K: ReST-EM is stronger at every K, largest gap at K = 1. - Transfer: MATH-trained models improve on GSM8K and Hungarian HS finals (the PaLM 2-L ReST-EM model surpasses all compared models except GPT-4); APPS-trained models **regress on HumanEval with more iterations**; Big-Bench Hard shows no major degradation. Per-benchmark base/SFT values are only in Figures 2–3 (not in fetched text). - Distillation: PaLM 2-L's single correct solutions distilled into PaLM 2-S beat human-written SFT despite fewer questions; multiple solutions per problem from 2-L ("Distill 2-L") beat PaLM 2-S's own ReST-EM data (Figure 6; exact values not in fetched text).

Method — equations. (Sections 2–3) Autoregressive policy $p_\theta(\mathbf{y} \mid \mathbf{x}) = \prod_{t=1}^{T} p_\theta(y_t \mid \mathbf{y}_{<t}, \mathbf{x})$. SFT loss, Eq. (1):

$$\mathcal{L}_{\mathrm{SFT}}(\theta) = -\mathbb{E}_{(\mathbf{x},\mathbf{y})\sim\mathcal{D}}\left[\sum_{t=1}^{T} \log p_\theta(y_t \mid \mathbf{y}_{1:t-1}, \mathbf{x})\right]$$

RL objective $\mathcal{L}_{\mathrm{RL}}(\theta) = \mathbb{E}_{\mathbf{x}\sim\mathcal{D}}\left[\mathbb{E}_{\mathbf{y}\sim p_\theta(\mathbf{y}\mid\mathbf{x})}[r(\mathbf{x},\mathbf{y})]\right]$ with deterministic sequence-level reward r. Introduce a binary optimality variable O with $p(O=1 \mid \mathbf{x},\mathbf{y}) \propto f(r(\mathbf{x},\mathbf{y}))$, f non-decreasing and non-negative. ELBO, Eq. (2):

$$\log p(O=1 \mid \mathbf{x}) \geq \mathbb{E}_{q(\mathbf{y}\mid\mathbf{x})}[\log p(O=1 \mid \mathbf{x},\mathbf{y})] - \mathrm{KL}[q(\mathbf{y} \mid \mathbf{x}) \parallel p_\theta(\mathbf{y} \mid \mathbf{x})] =: L(p_\theta, q)$$

E-step: $q^{t+1} = \arg\max_q L(p_{\theta^t}, q)$, with $q^{t+1}(\mathbf{y} \mid \mathbf{x}) \propto q^*(\mathbf{y} \mid \mathbf{x}) := p(O=1 \mid \mathbf{x},\mathbf{y})\, p_{\theta^t}(\mathbf{y} \mid \mathbf{x})$. M-step: $\theta^{t+1} := \arg\max_\theta L(p_\theta, q^{t+1}) = \arg\min_\theta \mathrm{KL}[q^{t+1} \parallel p_\theta] = \arg\min_\theta \sum_{\mathbf{y}} -q^{t+1}(\mathbf{y} \mid \mathbf{x}) \log p_\theta(\mathbf{y} \mid \mathbf{x})$. Monotonic improvement: $L(p_{\theta^{t+1}}, q^{t+1}) \geq L(p_{\theta^t}, q^{t+1}) \geq L(p_{\theta^t}, q^t)$. With non-negative reward and $f(r) = r$, Eq. (3):

$$\theta^{t+1} := \arg\max_\theta \mathbb{E}_{\mathbf{x}\sim\mathcal{D}}\left[\mathbb{E}_{\mathbf{y}\sim p_\theta^t(\mathbf{y}\mid\mathbf{x})}[r(\mathbf{x},\mathbf{y}) \log p_\theta(\mathbf{y} \mid \mathbf{x})]\right]$$

ReST-EM data collection and Improve objective:

$$\mathcal{D}_i = \{(\mathbf{x}^j, \mathbf{y}^j)\}_{j=1}^{N}, \quad \mathbf{x}^j \sim \mathcal{D},\ \mathbf{y}^j \sim p_\theta(\mathbf{y} \mid \mathbf{x}^j); \qquad J(\theta) = \mathbb{E}_{(\mathbf{x},\mathbf{y})\sim\mathcal{D}_i}[r(\mathbf{x},\mathbf{y}) \log p_\theta(\mathbf{y} \mid \mathbf{x})]$$

Binary rewards $r \in \{0,1\}$; with a threshold $f(r) = [r > \tau]$, any $\tau \in (0,1)$ gives identical Improve steps. There is no separate KL penalty: drift is limited by always fine-tuning the base model.

Method — algorithm. (Algorithm 1, ReST-EM)

```
Input: training set D (inputs); validation set D_val; loss L(x,y;theta); non-negative reward r(x,y); iterations
I; samples per context N
for i in 1..I:
    # Generate (E-step)
    D_i <- { (x^j, y^j) : x^j ~ D, y^j ~ p_theta(y | x^j) }_{j=1..N}      # current policy
    score each (x,y) in D_i with r(x,y)
    # Improve (M-step)
    theta <- base pretrained parameters                                    # NOT theta from iteration i-1
    while reward improves on D_val:
        theta <- theta - grad of -J(theta) on D_i   (next-token loss on model-generated targets only)
Output: policy p_theta
```

The paper: it "always fine tun[es] the base pretrained language model" to minimize drift and mitigate task-specific overfitting; task-specific performance is comparable to continuing from the last checkpoint (ReST), but transfer to held-out tasks is "substantially better" (Figure 7, HumanEval). Unlike ReST, human-written outputs are *not* mixed into the generated data.

Hyperparameters and setup. - Training problems: 7,500 (MATH), 2,342 (APPS Introductory). - Samples per question (E-step): 32 (MATH), 64 (APPS); sampling top-K = 40, temperature 0.7; **cap of 10 correct solutions per question** in the fine-tuning set. - Iterations: MATH up to 3; APPS up to 2. - Loss: next-token prediction on the model-generated solution only; few-shot prompt + question are the input. - Pass@K evaluation sampling: temperature 1.0, nucleus p = 0.95. - Fine-tuning steps, learning rate, batch size: not in fetched text (verify in paper Appendix).

Ablations that matter. Training accuracy rises roughly linearly with iterations but test accuracy does not; MATH gains are small after iteration 1; APPS regresses at iteration 2, attributed to its small training set. Multiple iterations beat one iteration with 3x samples (41.9 vs 40.3). 1,000 questions already give significant gains; 4,000 slightly below 2,000 (attributed to fine-tuning variance). Gains are largest on medium and hard MATH problems. ReST* with one correct sample per question outperforms SFT on human solutions for the same 5K questions.

Code / data / license. No code release; PaLM 2 via Google Cloud APIs. License not stated. Venue: Transactions on Machine Learning Research (04/2024).

Stated limitations. Needs a moderately sized set of problems/prompts per task; needs a manually designed or learned reward (ideally automatic); “requires some vigilance to avoid over-fitting” (gains flatten, small sets regress); may not close the gap to pass@K.

Use in our design. The weekly SFT is literally ReST-EM’s Improve step: reward = verifier verdict in {0,1}; keep at most 10 passing samples per task (prevents a handful of easy tasks from dominating); loss only on the completion; early-stop on held-out reward; **retrain the adapter from the frozen base each week** on the current week’s data plus replay, rather than stacking adapters, because that is the variant with better transfer and less drift. Watch the transfer set (ReST-EM saw HumanEval regress with APPS iterations) and stop after 2–3 passes over a small dataset. Distillation from the senior: sample several senior solutions per task, verify, SFT the worker on them — ReST-EM found multiple teacher solutions per problem beat the student’s own self-generated data.

PDF: https://arxiv.org/pdf/2312.06585

1.3 Direct Preference Optimization: Your Language Model is Secretly a Reward Model (Rafailov et al., 2023) — arXiv:2305.18290

Problem and claim. RLHF fits a reward model on human preferences and then optimizes the LM with RL (PPO) under a KL constraint to a reference policy — complex and unstable. DPO shows the KL-constrained reward maximization has a closed-form optimal policy, so the reward can be reparameterized in terms of the policy and the preference likelihood can be maximized directly with a binary cross-entropy loss: no explicit reward model, no sampling during training, no RL loop.

Headline results. Models up to 6B parameters: GPT-2-large (IMDb sentiment), the CarperAI GPT-J TL;DR SFT model (summarization), Pythia-2.8B (Anthropic HH dialogue). - IMDb: DPO dominates the reward-vs-KL frontier, above PPO even with ground-truth reward (Fig. 2 left). - TL;DR summarization, GPT-4 win rate vs reference summaries: DPO about 61% at temperature 0.0 vs about 57% for PPO at its best temperature; DPO stays robust as temperature rises while PPO degrades. Humans preferred DPO (temp 0.25) over PPO (temp 0) about 58% of the time. - Out-of-distribution CNN/DailyMail (GPT-4 win rate vs ground truth): DPO 0.36 (temp 0) / 0.31 (temp 0.25) vs PPO 0.26 / 0.23. - Anthropic-HH: DPO is the only computationally efficient method that improves over the preferred completions, roughly matching Best-of-128.

Method — equations. (Sections 3–4; symbols: x prompt; y_w, y_l preferred/dispreferred completions; r^* latent true reward; r_ϕ learned reward; σ logistic function; π_θ policy; π_{ref} reference policy, normally the SFT model; β KL strength; $Z(x)$ partition function.)

Bradley–Terry preference model, Eq. (1):

$$p^*\left(y_1 \succ y_2 \mid x\right) = \frac{\exp\left(r^*\left(x, y_1\right)\right)}{\exp\left(r^*\left(x, y_1\right)\right) + \exp\left(r^*\left(x, y_2\right)\right)}$$

Reward-model loss, Eq. (2):

$$\mathcal{L}_R\left(r_\phi, \mathcal{D}\right) = -\mathbb{E}_{\left(x, y_w, y_l\right) \sim \mathcal{D}}\left[\log \sigma\left(r_\phi\left(x, y_w\right) - r_\phi\left(x, y_l\right)\right)\right]$$

KL-constrained RL objective, Eq. (3):

$$\max_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D},\, y \sim \pi_\theta(y \mid x)}\left[r_\phi(x, y)\right] - \beta \mathbb{D}_{\text{KL}}\left[\pi_\theta(y \mid x) \parallel \pi_{\text{ref}}(y \mid x)\right]$$

Derivation step 1 — closed-form optimum of Eq. (3) for any reward r, Eq. (4):

$$\pi_r(y \mid x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r(x, y)\right), \qquad Z(x) = \sum_y \pi_{\text{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r(x, y)\right)$$

Step 2 — invert to express the reward through the policy, Eq. (5):

$$r(x, y) = \beta \log \frac{\pi_r(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x)$$

Step 3 — substitute into Eq. (1); $Z(x)$ cancels in the difference, Eq. (6):

$$p^*\left(y_1 \succ y_2 \mid x\right) = \frac{1}{1 + \exp\left(\beta \log \frac{\pi^*\left(y_2 \mid x\right)}{\pi_{\text{ref}}\left(y_2 \mid x\right)} - \beta \log \frac{\pi^*\left(y_1 \mid x\right)}{\pi_{\text{ref}}\left(y_1 \mid x\right)}\right)}$$

Step 4 — maximum likelihood of the preferences under this parameterization, the DPO loss, Eq. (7):

$$\mathcal{L}_{\text{DPO}}\left(\pi_\theta; \pi_{\text{ref}}\right) = -\mathbb{E}_{\left(x, y_w, y_l\right) \sim \mathcal{D}}\left[\log \sigma\left(\beta \log \frac{\pi_\theta\left(y_w \mid x\right)}{\pi_{\text{ref}}\left(y_w \mid x\right)} - \beta \log \frac{\pi_\theta\left(y_l \mid x\right)}{\pi_{\text{ref}}\left(y_l \mid x\right)}\right)\right]$$

Gradient (unnumbered in the fetched text, “Eq. 8” here) with implicit reward $\hat{r}_\theta\left(x, y\right) = \beta \log \frac{\pi_\theta\left(y \mid x\right)}{\pi_{\text{ref}}\left(y \mid x\right)}$:

$$\nabla_\theta \mathcal{L}_{\text{DPO}} = -\beta \mathbb{E}_{\left(x, y_w, y_l\right) \sim \mathcal{D}}[\sigma\left(\hat{r}_\theta\left(x, y_l\right) - \hat{r}_\theta\left(x, y_w\right)\right)[\nabla_\theta \log \pi_\theta\left(y_w \mid x\right) - \nabla_\theta \log \pi_\theta\left(y_l \mid x\right)]]$$

The sigmoid weight is large when the implicit reward orders the pair wrongly — “how incorrectly the implicit reward model orders the completions, accounting for the strength of the KL constraint”; dropping the weighting makes the model degenerate. Plackett–Luce extension for a ranking τ of K completions (Appendix A.3, Eq. 20):

$$\mathcal{L}_{\text{DPO}}\left(\pi_\theta, \pi_{\text{ref}}\right) = -\mathbb{E}\left[\log \prod_{k=1}^{K} \frac{\exp\left(\beta \log \frac{\pi_\theta\left(y_{\tau(k)} \mid x\right)}{\pi_{\text{ref}}\left(y_{\tau(k)} \mid x\right)}\right)}{\sum_{j=k}^{K} \exp\left(\beta \log \frac{\pi_\theta\left(y_{\tau(j)} \mid x\right)}{\pi_{\text{ref}}\left(y_{\tau(j)} \mid x\right)}\right)}\right]$$

reduces to Bradley–Terry at $K = 2$.

Method — algorithm. (the “DPO outline”) 1. For each prompt x, sample completion pairs $y_1, y_2 \sim \pi_{\text{ref}}\left(\cdot \mid x\right)$ and label preferences to build an offline dataset $\mathcal{D} = \{\left(x, y_w, y_l\right)\}$. 2. Set $\pi_{\text{ref}} = \pi_{\text{SFT}}$ when available. If not, “we initialize π_{ref} by maximizing likelihood of preferred completions” — this “helps mitigate the distribution shift between the true reference distribution which is unavailable” and the one DPO uses. 3. Minimize Eq. (7) over π_θ with π_{ref} frozen; evaluate every 100 steps until convergence (Section 6.1).

Hyperparameters and setup. (Appendix B) $\beta = 0.1$ by default; $\beta = 0.5$ for TL;DR summarization. Batch size 64. RMSprop, learning rate 1e-6, warmed up linearly from 0 over 150 steps. IMDb: SFT for 1 epoch; reward model 3 epochs with best-validation checkpoint. Completions sampled at temperatures from 0.0 to 1.0 for evaluation. Total DPO steps/epochs: not in fetched text.

Ablations that matter. DPO robust across sampling temperature where PPO is not; generalizes OOD better than PPO on CNN/DailyMail; small late drop in Fig. 3 right may be over-optimization.

Code / data / license. Reference implementation: https://github.com/eric-mitchell/direct-preference-optimization (found by search; not in fetched paper text). SFT summarization model https://huggingface.co/CarperAI/openai_summarize_tldr_sft. Paper license on the HTML page: CC BY 4.0.

Stated limitations. (Section 7) OOD generalization of DPO policies vs explicit-reward methods; self-labeling unlabeled prompts with the DPO policy untested; reward over-optimization under DPO; scaling to much larger models; GPT-4 win rates are prompt-sensitive.

Use in our design. DPO is the pair-learning step: y_w = the verified (tests pass) or human-corrected trajectory, y_l = the failed trajectory for the same task. π_{ref} = base + this week’s SFT adapter (so SFT happens first, then DPO from it, as in the paper’s pipeline). Start at $\beta = 0.1$. Log-probabilities must be summed over the completion tokens only (prompt masked). Since the pairs are sampled by the worker itself, the reference is the on-distribution SFT model, which is the case the paper recommends.

PDF: https://arxiv.org/pdf/2305.18290 — Code: https://github.com/eric-mitchell/direct-preference-optimization

1.4 Absolute Zero: Reinforced Self-play Reasoning with Zero Data (Zhao et al., 2025) — arXiv:2505.03335

Problem and claim. RL with verifiable rewards (RLVR) still depends on expert-curated question sets. Absolute Zero: a single model **proposes** its own code-reasoning tasks, **solves** them, and improves, with a Python executor as the only verifier and no external data. The Absolute Zero Reasoner (AZR) “outperforms all previous models” by 1.8 average points over zero-setting models trained on in-domain curated data.

Headline results. (Table 1, Qwen2.5-7B family; greedy decoding)

Model	HEval+	MBPP+	LCB v1-5	AIME24	AIME25	AMC	MATH500	Minerva	Olympiad	Code Avg	Math Avg	Overall
Qwen2.5-7B	73.2	65.3	17.5	6.7	3.3	37.5	64.8	25.0	27.7	52.0	27.5	39.8
Qwen2.5-7B-Coder	80.5	69.3	19.9	6.7	3.3	40.0	54.0	17.3	21.9	56.6	23.9	40.2
AZR-Base-7B	71.3	69.1	25.3	13.3	13.3	52.5	74.4	38.2	38.5	55.2 (+3.2)	38.4 (+10.9)	46.8 (+7.0)
AZR-Coder-7B	83.5	69.6	31.7	20.0	10.0	57.5	72.6	36.4	38.2	61.6 (+5.0)	39.1 (+15.2)	50.4 (+10.2)

Scaling (AZR-Coder overall gains): 3B +5.7, 7B +10.2, 14B +13.2. Cross-domain: expert code models improved math by only 0.65 on average after RLVR, AZR-Base-7B +10.9 and AZR-Coder-7B +15.2.

Method — equations. (Section 2–3) SFT, Eq. (1): $\mathcal{L}_{\mathrm{SFT}}(\theta) = -\mathbb{E}_{(x,c^\star,y^\star)\sim\mathcal{D}} \log \pi_\theta(c^\star, y^\star \mid x)$. RLVR, Eq. (2): $J_{\mathrm{RLVR}}(\theta) = \mathbb{E}_{(x,y^\star)\sim\mathcal{D},\,(c,y)\sim\pi_\theta(\cdot\mid x)}[r(y,y^\star)]$. Absolute Zero joint objective, Eq. (3):

$$\mathcal{J}(\theta) := \max_\theta \mathbb{E}_{z\sim p(z)}[\mathbb{E}_{(x,y^\star)\sim f_e(\cdot\mid\tau),\,\tau\sim\pi_\theta^{\mathrm{propose}}(\cdot\mid z)}[\lambda\, r_e^{\mathrm{propose}}(\tau,\pi_\theta) + \mathbb{E}_{y\sim\pi_\theta^{\mathrm{solve}}(\cdot\mid x)}[r_e^{\mathrm{solve}}(y,y^\star)]]]$$

θ shared parameters; $z\sim p(z)$ conditioning seed (in practice sampled past tasks); τ proposed task; f_e the environment (executor) that validates τ into $(x,y^\star)$; $\lambda\geq 0$ trades learnability against solving. Proposer learnability reward, Eq. (4), with $\bar{r}_{\mathrm{solve}} = \frac{1}{G}\sum_{i=1}^{G} r_{\mathrm{solve}}^{(i)}$ over G Monte-Carlo solver rollouts at non-zero temperature:

$$r_{\mathrm{propose}} = \begin{cases} 0, & \bar{r}_{\mathrm{solve}} = 0 \\ 1-\bar{r}_{\mathrm{solve}}, & \text{otherwise} \end{cases}$$

(trivially easy or unsolvable tasks earn nothing; moderately difficult tasks earn most). Solver reward, Eq. (5): $r_{\mathrm{solve}} = \mathbb{I}(y = y^\star)$ with Python value equality. Format-aware composite reward, Eq. (6):

$$R(y_\pi) = \begin{cases} r_{\mathrm{role}}, & \text{correctly formatted, role} \in \{\mathrm{propose},\mathrm{solve}\} \\ -0.5, & \text{wrong but well-formatted} \\ -1, & \text{formatting errors} \end{cases}$$

Task-Relative REINFORCE++ (TRR++), Section 3.3.5 — six baselines, one per (task, role):

$$A^{\mathrm{norm}}_{\mathrm{task,role}} = \frac{r-\mu_{\mathrm{task,role}}}{\sigma_{\mathrm{task,role}}}, \quad \mathrm{task}\in\{\mathrm{ind},\mathrm{ded},\mathrm{abd}\},\ \mathrm{role}\in\{\mathrm{propose},\mathrm{solve}\}$$

REINFORCE++ uses one global baseline, GRPO a per-question baseline; TRR++ interpolates with per-(task, role) means and standard deviations.

Tasks are triplets (p,i,o) with $o = p(i)$:

Mode	Proposer emits	Solver gets → predicts	Check
Deduction	(p,i); env computes o	$(p,i)\to o_\pi$	$o_\pi = o^\star$
Abduction	(p,i); env computes o	$(p,o)\to i_\pi$	$p(i_\pi) = p(i^\star)$ (programs need not be bijective)
Induction	program p + N new inputs + message m; env computes outputs	half of the (i^n,o^n) pairs + m → p_π	p_π correct on all held-out inputs

Method — algorithm. (Algorithm 1 and Section 3.3, paraphrased from fetched text)

```
# 3.3.1 seeding (no gradient updates)
seed = identity triplet  (def f(x): return x ; "Hello World" -> "Hello World"), used only if the buffer is empty
base LLM generates (p, i) pairs for deduction/abduction; execute; store triplets until |D_seed| = B * S   (S = 4)
induction buffer: sample programs from D_seed, generate input sets + message until |D^0_induction| = B * S
strip global variables and comments from seed programs
for t in 1..T:
    # propose
    for each of B batch slots:
        deduction/abduction: sample K past triplets uniformly from the buffer as in-context references,
                             ask for a *different* task; validate; add valid triplets to the buffer regardless of
reward
        induction:           sample one program from D_abd ∪ D_ded; proposer generates N inputs + message;
validate; add
        if fewer than B valid proposals: fill the batch by sampling previously validated tasks
    # validate (3.3.3)
        program integrity: runs on the input without error and returns a value
        program safety:    forbidden modules os, sys, shutil, ... (full list Figure 10)
        determinism:       run j = 2 times, outputs must match
        induction:         solver sees N//2 pairs + message; the other half is held out for checking
    # solve
    for each task type: build tasks from buffers; solver answers; verify per the table above
    # rewards and update
    r_propose from G solver rollouts (Eq. 4); r_solve (Eq. 5); format penalties (Eq. 6)
    RL update with TRR++ advantages
```

Hyperparameters and setup. Qwen2.5-7B / 7B-Coder main; 3B/14B Coder, 14B base, Llama-3.1-8B additional. Batch 64 × 6 (2 roles × 3 task types). AdamW, constant LR 1e-6. S = 4; j = 2 determinism runs. Training to about 250 steps in Fig. 6(a); 7B/14B keep improving past 200 steps. Eval: greedy; pass@k with temperature 0.6, top-p

0.95, 16k max tokens. **K (in-context references), G (solver rollouts), λ, N (induction inputs), max lengths, KL/entropy coefficients: not in fetched text — verify in paper Appendix "Table 3 hyperparameters".** Hardware per README: 3B needs 2 × 80 GB GPUs, 7B/8B 4 × 80 GB, 14B 8 × 80 GB.

Ablations that matter. (Table 2, AZR-Base-7B overall avg 46.8) Deduction only 43.3 (−3.5); without induction 43.8 (−3.0); without references K = 0 43.8 (−3.0; math −5, code −1); train solver only (no proposer training) 45.4 (−1.4).

Code / data / license. https://github.com/LeapLabTHU/Absolute-Zero-Reasoner (MIT per repo sidebar); project page https://andrewzh112.github.io/absolute-zero-reasoner/; models https://huggingface.co/collections/andrewzh/absolute-zero-reasoner-68139b2bca82afb00bc69e5b. Training code is a veRL fork with vLLM rollouts; optional Sandbox-Fusion executor.

Stated limitations. Safety of self-improving systems unaddressed; an "uh-oh moment" on Llama-3.1-8B ("The aim is to outsmart all these groups of intelligent machines…", Fig. 34) — oversight still needed. Future work: other environments, exploration rewards, better learning-progress estimates.

Use in our design. We do not run online RL, but the weekly job can use AZR's *task factory* when a domain's weekly data is thin: the senior (or worker) proposes deduction/abduction/induction triplets over real repo code, the sandbox validates (integrity, forbidden modules, determinism j = 2), the worker solves G times, and tasks with $0 < \bar{r}_{\text{solve}} < 1$ (the learnability band) are kept; passing solutions feed SFT, pass/fail pairs feed DPO. Format penalties (−0.5 / −1) become hard filters. The executor sandbox is mandatory; the safety filter list is the minimum.

PDF: https://arxiv.org/pdf/2505.03335 — Code: https://github.com/LeapLabTHU/Absolute-Zero-Reasoner

1.5 LoRA: Low-Rank Adaptation of Large Language Models (Hu et al., 2021) — arXiv:2106.09685

Problem and claim. Full fine-tuning stores a full copy per task; adapters add sequential compute and inference latency; prefix tuning is hard to optimize and eats sequence length. LoRA freezes W_0 and learns the update as a low-rank product $\Delta W = BA$ (hypothesis: adaptation updates have low intrinsic rank). After training BA merges into W_0, so inference adds no latency. Trainable parameters drop up to 10,000x and GPU memory about 3x versus Adam full fine-tuning of GPT-3 175B.

Headline results. - GLUE average: RoBERTa-base FT 125.0M params 86.4 vs LoRA 0.3M 87.2; RoBERTa-large FT 355.0M 88.9 vs LoRA 0.8M 89.0; DeBERTa-XXL FT 1500.0M 91.1 vs LoRA 4.7M 91.3. - GPT-2 E2E NLG (BLEU/NIST/MET/ROUGE-L/CIDEr): Medium FT 354.92M 68.2/8.62/46.2/71.0/2.47 vs LoRA 0.35M 70.4/8.85/46.8/71.8/2.53; Large FT 774.03M 68.5/8.78/46.0/69.9/2.45 vs LoRA 0.77M 70.4/8.89/46.8/72.0/2.47. - GPT-3 175B (WikiSQL acc / MNLI-m acc / SAMSum R1/R2/RL): FT 175,255.8M 73.8/89.5/52.0-28.0-44.5; LoRA 4.7M 73.4/91.7/53.8-29.8-45.9; LoRA 37.7M 74.0/91.6/53.4-29.2-45.1; AdapterH 40.1M 73.2/91.5/53.2-29.0-45.1; prefix-layer 20.2M 70.1/89.5/50.8-27.3-43.5. - GPT-3 175B training: VRAM 1.2 TB to 350 GB; checkpoint 350 GB to 35 MB (r = 4, W_q, W_v); 25% training speedup.

Method — equations. Full fine-tuning, Eq. (1): $\max_{\Phi} \sum_{(x,y) \in \mathcal{Z}} \sum_{t=1}^{|y|} \log P_{\Phi}\left(y_t \mid x, y_{<t}\right)$. LoRA objective, Eq. (2), with $\Delta\Phi = \Delta\Phi(\Theta)$ and $|\Theta| \ll |\Phi_0|$:

$$\max_{\Theta} \sum_{(x,y) \in \mathcal{Z}} \sum_{t=1}^{|y|} \log p_{\Phi_0 + \Delta\Phi(\Theta)}\left(y_t \mid x, y_{<t}\right)$$

Section 4.1, Eq. (3):

$$W_0 + \Delta W = W_0 + BA, \quad W_0 \in \mathbb{R}^{d \times k},\ B \in \mathbb{R}^{d \times r},\ A \in \mathbb{R}^{r \times k},\ r \ll \min(d, k); \qquad h = W_0 x + \Delta W x = W_0 x + BAx$$

Initialization: A random Gaussian, $B = 0$, so $\Delta W = 0$ at start. Scaling: $\Delta W x$ is multiplied by α / r, α a constant in r; "we simply set α to the first r we try and do not tune it" (tuning α is then roughly the same as tuning the learning rate). Merge for inference: $W = W_0 + BA$; switch tasks by subtracting BA and adding $B'A'$. Parameter count with $\widehat{L}_{\text{LoRA}}$ adapted matrices of size $d_{\text{model}} \times d_{\text{model}}$:

$$|\Theta| = 2 \times \widehat{L}_{\text{LoRA}} \times d_{\text{model}} \times r$$

Method — algorithm. Which matrices (Section 4.2): only the self-attention weights W_q, W_k, W_v, W_o are considered; MLP frozen; most experiments adapt only W_q and W_v. Training: freeze W_0, train A, B with the task loss; no optimizer state for frozen weights (the VRAM saving).

Hyperparameters and setup. (Appendix D) - RoBERTa-base LoRA: $r_q = r_v = 8$, $\alpha = 8$, AdamW, linear schedule, warmup ratio 0.06, max seq 512; per task batch 16–32, epochs 25–80, LR 4e-4–5e-4. - RoBERTa-large LoRA: $r_q = r_v = 8$, $\alpha = 16$; batch 4–8, epochs 10–30, LR 2e-4–4e-4. - DeBERTa-XXL LoRA: $r_q = r_v = 8$, $\alpha = 8$, warmup 0.1; LR 6e-5–2e-4; weight decay 0–0.1. - GPT-2 (E2E/WebNLG/DART): AdamW, linear, batch 8, 5 epochs, 500 warmup steps,

LR 2e-4, $r_q = r_v = 4$, $\alpha = 32$, weight decay 0.01/0.01/0.0, dropout 0.1/0.1/0.0, label smoothing 0.1/0.1/0.0; beam 10. - GPT-3 175B: AdamW, batch 128, 2 epochs, weight decay 0.1, warmup 250,000 tokens, linear, LR 2e-4, $r_q = r_v = 8$ (37.7M) or $r_q = r_v = 1$ / $r_v = 2$ (4.7M); seq len 384/768/2048. α for GPT-3: not in fetched text.

Ablations that matter. Table 5 (GPT-3, 18M budget): W_q r=8 70.4/91.0; W_k r=8 70.0/90.8; W_v r=8 73.0/91.0; W_o r=8 73.2/91.3; W_q, W_k r=4 71.4/91.3; W_q, W_v r=4 73.7/91.3; all four r=2 73.7/91.7 (WikiSQL/MNLI) — adapting more matrices at lower rank beats one matrix at higher rank. Table 6 (rank): W_q, W_v WikiSQL r=1 73.4, r=2 73.3, r=4 73.7, r=8 73.8, r=64 73.5; all four r=1 74.1 … r=64 73.9 — "a rank as small as one" is competitive when adapting W_q, W_v; W_q alone needs larger r. Subspace: top singular directions of ΔW for r=8 and r=64 overlap strongly; ΔW amplifies directions W_0 does not emphasize, amplification factor $\| \Delta W \|_F / \| U^\top W_0 V^\top \|_F = 6.91/0.32 \approx 21.5$ at r=4, about 2 at r=64.

Code / data / license. https://github.com/microsoft/LoRA. Code license not stated in paper (check repo); dataset licenses listed (WikiSQL BSD-3, E2E/WebNLG CC BY-NC-SA 4.0, DART MIT, SAMSum CC BY-NC-ND 4.0).

Stated limitations. "it is not straightforward to batch inputs to different tasks with different A and B in a single forward pass" (Section 4.2); "We mostly depend on heuristics to select the weight matrices to apply LoRA to" (Section 8); "we do not expect a small r to work for every task or dataset" (footnote 6; e.g. a language unseen in pretraining).

Use in our design. One adapter = one domain pocket: a small safetensors file ($2\widehat{Ldr}$ parameters) that is swapped by subtracting/adding BA, or merged for serving. Follow the paper's α convention (fix α, tune LR). Spread rank over more matrices rather than piling rank on one (Table 5). Start r small; QLoRA (next) shows that for decoder LLMs covering *all* linear layers matters more than rank.

PDF: https://arxiv.org/pdf/2106.09685 — Code: https://github.com/microsoft/LoRA

1.6 QLoRA: Efficient Finetuning of Quantized LLMs (Dettmers et al., 2023) — arXiv:2305.14314

Problem and claim. 16-bit fine-tuning of LLaMA 65B needs over 780 GB of GPU memory; existing quantization works for inference but breaks training. QLoRA back-propagates through a frozen **4-bit NormalFloat (NF4)** base model into LoRA adapters, with **Double Quantization** of the quantization constants and **Paged Optimizers** for memory spikes, and matches 16-bit full fine-tuning and 16-bit LoRA on the benchmarks tested.

Headline results. - Memory: 65B fine-tuning from >780 GB to <48 GB on one GPU. Guanaco memory (Table 1/6): 7B 5–6 GB, 13B 10 GB, 33B 21 GB, 65B 41 GB; Figure 6 lists 7B 6.9 GB, 13B 11.3 GB, 33B 24.7 GB, 65B 45.0 GB. - Guanaco 65B: 99.3% of ChatGPT on the Vicuna benchmark; 33B: 97.8%, trainable on a 24 GB consumer GPU in under 12 h; 7B: 87.0%. - MMLU 5-shot (Table 4; Alpaca / FLAN v2): BF16 7B 38.4/45.6, 13B 47.2/50.6, 33B 57.7/60.5, 65B 61.8/62.5, mean 53.0; Float4 mean 52.2; NF4 + DQ 7B 39.0/44.5, 13B 47.5/50.7, 33B 57.3/59.2, 65B 61.8/63.9, mean 53.1 — NF4+DQ matches 16-bit, Float4 trails. - Pile Common Crawl perplexity (Table 2, lower better): Int4 34.34, Float4 E2M1 31.07, Float4 E3M0 29.48, NF4 + DQ 27.41.

Method — equations. Block-wise absmax quantization, Eq. (1), and dequantization, Eq. (2):

$$X^{\text{Int8}} = \text{round}\left(\frac{127}{\text{absmax}\left(X^{\text{FP32}}\right)} X^{\text{FP32}}\right) = \text{round}\left(c^{\text{FP32}} X^{\text{FP32}}\right), \qquad \text{dequant}\left(c^{\text{FP32}}, X^{\text{Int8}}\right) = \frac{X^{\text{Int8}}}{c^{\text{FP32}}} = X^{\text{FP32}}$$

c is the per-block quantization constant; blocks of 64 weights for the 4-bit base. NormalFloat quantiles, Eq. (4), Q_X the quantile function of $\mathcal{N}(0,1)$, k bits:

$$q_i = \frac{1}{2}\left(Q_X\left(\frac{i}{2^k+1}\right) + Q_X\left(\frac{i+1}{2^k+1}\right)\right)$$

then normalize the q_i into $[-1,1]$ and the weights per block into $[-1,1]$ by absmax. Asymmetric construction: 2^{k-1} quantiles for the negative half and $2^{k-1}+1$ for the positive half, union with one of the two zeros removed, so NF4 has an exact zero (16 levels). Double quantization: first-level constants c_1^{FP32} (block 64) are themselves quantized to FP8 constants c_2 with block size 256; cost drops from $32/64 = 0.5$ bits per parameter to

$$\frac{8}{64} + \frac{32}{64 \cdot 256} = 0.127 \text{ bits per parameter}$$

(saving about 0.373 bits/parameter). QLoRA linear layer, Eq. (5), and double dequantization, Eq. (6):

$$Y^{\text{BF16}} = X^{\text{BF16}} \,\text{doubleDequant}\left(c_1^{\text{FP32}}, c_2^{k\text{-bit}}, W^{\text{NF4}}\right) + X^{\text{BF16}} L_1^{\text{BF16}} L_2^{\text{BF16}}$$

$$\text{doubleDequant}\left(c_1^{\text{FP32}}, c_2^{k\text{-bit}}, W^{k\text{-bit}}\right) = \text{dequant}\left(\text{dequant}\left(c_1^{\text{FP32}}, c_2^{k\text{-bit}}\right), W^{4\text{-bit}}\right) = W^{\text{BF16}}$$

$L_1 \in \mathbb{R}^{h \times r}$, $L_2 \in \mathbb{R}^{r \times o}$ the LoRA factors. Storage NF4, compute BF16: each weight is dequantized to BF16 before the matmul; gradients flow through the dequantized weights to the inputs but only the LoRA parameters receive weight gradients. Paged optimizers: optimizer state in NVIDIA unified memory, paged CPU↔GPU automatically when GPU memory runs out.

Method — algorithm. (1) Quantize every linear weight of the frozen base to NF4 in blocks of 64 with FP32 absmax constants; (2) quantize the constants to FP8 in blocks of 256 (DQ); (3) attach LoRA to all linear layers; (4) forward with Eq. (5) in BF16; (5) backward only into L_1, L_2; (6) optimizer states paged; (7) gradient checkpointing on.

Hyperparameters and setup. (Appendix B.2, Table 9) - LoRA r = 64, α = 16, on **all linear layers**; LoRA dropout 0.1 (≤13B), 0.05 (33B/65B); NF4 + DQ; BF16 compute; Adam β_2 = 0.999; max grad norm 0.3; constant LR; group-by-length; gradient checkpointing; paged optimizer. - 7B/13B: batch 16, LR 2e-4, 10,000 steps (most datasets) / 1,875 (OASST1) / 4,000 (Longform); source/target length 384/128 (512 target OASST1, 768 HH-RLHF, 512/1024 Longform). - 33B: batch 32 (16 OASST1), LR 1e-4, 5,000 / 1,875 / 2,343 steps. 65B: batch 64 (16 OASST1, 32 Longform), LR 1e-4, 2,500 / 1,875 / 2,343 steps. - LoRA search space (Appendix A.1): dropout {0.0, 0.05, 0.1}; r in 8..256. Warmup ratio, weight decay, optimizer name beyond “paged”: not in fetched text.

Ablations that matter. Figure 2: “LoRA on all linear transformer block layers are required to match full finetuning performance”; query/value only does not match. Figure 4 / A.1: “LoRA r is unrelated to final performance if LoRA is used on all layers”. NF4 > Float4 > Int4 at equal bits (Table 2, Fig. 3). Data quality matters more than size (OASST1 9k beats FLAN v2 450k for chat).

Code / data / license. https://github.com/artidoro/qlora and https://github.com/TimDettmers/bitsandbytes (4-bit CUDA kernels). License not stated in fetched text (Guanaco weights inherit the LLaMA license; check repos).

Stated limitations. (Section 8) “did not establish that QLoRA can match full 16-bit finetuning performance at 33B and 65B scales”; benchmarks limited to MMLU, Vicuna, OA; limited responsible-AI evaluation (CrowS only); only 4-bit + LoRA tested.

Use in our design. This is the memory recipe for the 9B worker in 12 GB: NF4 + DQ base (about 4.127 bits/parameter for the quantized matrices), LoRA on all linear layers, BF16 compute, paged optimizer, gradient checkpointing, batch 1 with gradient accumulation and length-grouped sequences. Start from r = 16–64, α = 16, dropout 0.1, LR 2e-4 (the 7B/13B setting), max grad norm 0.3, constant LR. The 7B (6.9 GB) and 13B (11.3 GB) measurements bracket a 9B at a few GB of headroom under 12 GB at QLoRA’s sequence lengths (384+128 / up to 1024) — longer coding traces push activation memory up, so sequence length is the knob to cut first.

PDF: https://arxiv.org/pdf/2305.14314 — Code: https://github.com/artidoro/qlora

1.7 Distilling the Knowledge in a Neural Network (Hinton et al., 2015) — arXiv:1503.02531

Problem and claim. Ensembles and large regularized nets predict well but are too expensive to deploy. The knowledge of a model is the learned input-to-output mapping, not its parameters; a small model can absorb it by matching the large model’s **softened** output distribution produced with a high softmax temperature (“soft targets”), which carries the “dark knowledge” of relative probabilities among wrong classes. Matching logits is a special case.

Headline results. - MNIST: large net (2 × 1200 units, dropout, jitter) 67 test errors; small net (2 × 800, no regularization) 146 errors; same small net trained on soft targets at T = 20: 74 errors. With all 3s removed from the transfer set: 206 errors (133 of the 1010 test threes); after raising the bias of class 3 by 3.5: 109 errors, 14 on 3s (about 98.6% of unseen 3s correct). - Speech (Table 1): baseline 58.9% frame accuracy / 10.9% WER; 10x ensemble 61.1% / 10.7%; distilled single model 60.8% / 10.7% (recovers >80% of the ensemble’s frame-accuracy gain). - JFT (Table 3): baseline 43.1% conditional / 25.0% top-1; baseline + 61 specialists 45.9% / 26.1% (4.4% relative). - Soft targets as regularizer (Table 5, 3% of speech data): hard targets 44.5% (needs early stopping) vs soft targets 57.0% (no early stopping); full-data baseline 58.9%.

Method — equations. Softmax with temperature, Eq. (1); z_i student logit for class i, T temperature (1 is normal):

$$q_i = \frac{\exp\left(z_i/T\right)}{\sum_j \exp\left(z_j/T\right)}$$

Cross-entropy gradient against the teacher’s soft targets p_i (teacher logits v_i at the same T), Eq. (2):

$$\frac{\partial C}{\partial z_i} = \frac{1}{T}\left(q_i - p_i\right) = \frac{1}{T}\left(\frac{e^{z_i/T}}{\sum_j e^{z_j/T}} - \frac{e^{v_i/T}}{\sum_j e^{v_j/T}}\right)$$

High-temperature expansion, Eq. (3), N the number of classes:

$$\frac{\partial C}{\partial z_i} \approx \frac{1}{T}\left(\frac{1+z_i/T}{N+\sum_j z_j/T} - \frac{1+v_i/T}{N+\sum_j v_j/T}\right)$$

With zero-mean logits per transfer case ($\sum_j z_j = \sum_j v_j = 0$), Eq. (4):

$$\frac{\partial C}{\partial z_i} \approx \frac{1}{NT^2}\left(z_i - v_i\right)$$

so high-temperature distillation equals matching logits (minimizing $\frac{1}{2}\left(z_i - v_i\right)^2$). Because the soft-target gradients scale as $1/T^2$, the paper says to multiply the soft-target term by T^2 when combining it with the hard-label loss so the relative contribution stays constant as T changes. The combined objective (stated in words, no numbered equation; α is our notation):

$$\mathcal{L} = \alpha T^2 \,\mathrm{CE}\left(p^{(T)}, q^{(T)}\right) + \left(1-\alpha\right)\,\mathrm{CE}\left(y_{\mathrm{hard}}, q^{(1)}\right)$$

with “considerably lower weight” on the hard-label term; the speech experiments used relative weight 0.5 on the hard cross-entropy and tried T in {1, 2, 5, 10}.

Method — algorithm. 1. Train (or take) the cumbersome model. 2. For each transfer example compute the teacher’s softmax at temperature T. 3. Train the student with the same T on the soft-target cross-entropy (times T^2) plus a lower-weighted hard-label cross-entropy at $T = 1$. 4. Deploy the student at $T = 1$. Transfer set can be unlabeled or the original training set.

Hyperparameters and setup. T = 20 for the 800-unit MNIST student; T ≥ 8 all similar for students with ≥ 300 hidden units; T = 2.5–4 significantly better for a 30-unit student — “when the distilled model is much too small to capture the cumbersome model’s knowledge, intermediate temperatures work best”. Speech: 8 × 2560 ReLU, 14,000-way softmax, about 85M params, 2000 h / 700M examples; ensemble of 10 differing only by initialization. Specialists initialized from the generalist, trained on enriched subsets with a dustbin class.

Ablations that matter. Soft targets with 3% of the data reach 57.0% vs 44.5% with hard targets (regularization effect). Omitted-class experiment shows soft targets transfer knowledge about classes never seen in the transfer set.

Code / data / license. None (JFT internal). Venue: NIPS 2014 Deep Learning Workshop (not on the abs page).

Stated limitations. Not yet shown that specialists’ knowledge can be distilled back into one net; WER gain smaller than frame-accuracy gain (objective mismatch); the class-3 bias was tuned on the test set; soft targets against specialist overfitting still being explored.

Use in our design. Token-level soft-target distillation from the senior requires the senior’s full-vocabulary logits per token and a **shared tokenizer/vocabulary** with the worker — only use it when the senior and worker share a vocabulary; then the loss above at T in 2–4 (worker is much smaller) on the senior’s verified trajectories, soft term scaled by T^2, hard term = the verified trajectory tokens at weight ≤ 0.5. Otherwise use sequence-level distillation (ReST-EM Figure 6): sample from the senior, verify, SFT the worker — which also works across tokenizers.

PDF: https://arxiv.org/pdf/1503.02531

1.8 Self-Rewarding Language Models (Yuan et al., 2024; ICML 2024) — arXiv:2401.10020

Problem and claim. RLHF/DPO depend on human preference data and a frozen reward model, a ceiling on the policy. A Self-Rewarding LM both follows instructions and judges its own candidate responses with an LLM-as-a-Judge prompt; preference pairs built from its own scores train it with **iterative DPO**, and both instruction following and judging improve across iterations.

Headline results. Llama 2 70B base. Seed data: IFT 3,200 Open Assistant examples; EFT 1,630 train / 541 eval judge examples. - AlpacaEval 2.0 win rate vs GPT-4 Turbo: M1 9.94%, M2 15.38%, M3 20.44% (Claude 2 17.19%, Gemini Pro 16.85%, GPT-4 0613 15.76%). - Head-to-head (GPT-4 judge): M2 vs M1 55.5% wins / 11.7% losses / 32.8% ties; M3 vs M2 47.7% / 12.5%; M2 vs SFT baseline 49.2% / 14.5%; M3 vs SFT baseline 62.5% / 9.8%. - Reward-modeling pairwise accuracy vs human rankings: SFT baseline 65.1%, M1 78.7%, M2 80.4%, M3 81.7% (5-best %, exact match, Spearman, Kendall also improve; exact match 14.3% for M2 and M3). - MT-Bench: SFT 6.85, M1 6.78, M2 7.01, M3 7.25. - AIFT sizes: 3,964 pairs (to train M2), 6,942 pairs (to train M3).

Method — equations. No new loss; the DPO loss (1.3, Eq. 7) is applied iteratively. Reward of a candidate = judge score $r \in \{0, ..., 5\}$ averaged over 3 sampled judgments. Definitions: **IFT** instruction fine-tuning data (human prompt–response pairs); **EFT** evaluation fine-tuning data (judge prompt + chain-of-thought justification + score, teaches the judge task); **AIFT** AI-feedback training data (self-judged preference pairs).

Method — algorithm. (Section 2; main-experiment settings)

```
M_0 <- base pretrained model
M_1 <- SFT(M_0, IFT ∪ EFT)
for t in 1..T:                                  # paper: T = 2 (M_2, M_3)
    # Self-instruction creation
    prompts <- few-shot generate new prompts x_i (8 demos: 6 from IFT, 2 from previously generated; main runs use
a fixed Llama-2-Chat-70B generator)
    for each x_i:
        y_i^1..y_i^N <- sample N = 4 responses from M_t at temperature 0.7, top-p 0.9
        for each y_i^n: r_i^n <- mean of 3 judge samples of M_t with the LLM-as-a-Judge prompt (same sampling
settings)
    # AI feedback data
    AIFT(M_t) <- { (x_i, y_w = argmax_n r_i^n, y_l = argmin_n r_i^n) : max != min }     # ties discarded
    M_{t+1} <- DPO(M_t, AIFT(M_t))              # pi_ref = M_t
```

Alternative (ablated, did not help): add responses scored 5/5 as extra SFT data — 11,254 added examples gave 29% wins vs 30% losses against the SFT baseline.

LLM-as-a-Judge prompt (Figure 2), assembled from verbatim fetched segments:

```
Review the user's question and the corresponding response using the additive 5-point scoring system described
below. Points are accumulated based on the satisfaction of each criterion:
- Add 1 point if the response is relevant and provides some information related to the user's inquiry, even if it
is incomplete or contains some irrelevant content.
- Add another point if the response addresses a substantial portion of the user's question, but does not
completely resolve the query or provide a direct answer.
- Award a third point if the response answers the basic elements of the user's question in a useful way,
regardless of whether it seems to have been written by an AI Assistant or if it has elements typically found in
blogs or search results.
- Grant a fourth point if the response is clearly written from an AI Assistant's perspective, addressing the
user's question directly and comprehensively, and is well-organized and helpful, even if there is slight room for
improvement in clarity, conciseness or focus.
- Bestow a fifth point for a response that is impeccably tailored to the user's question by an AI Assistant,
without extraneous information, reflecting expert knowledge, and demonstrating a high-quality, engaging, and
insightful answer.

User: <INSTRUCTION_HERE>
<response><RESPONSE_HERE></response>

After examining the user's instruction and the response:
- Briefly justify your total score, up to 100 words.
- Conclude with the score using the format: "Score: <total points>"
Remember to assess from the AI Assistant perspective, utilizing web search knowledge as necessary.
```

(The two closing bullets and the “Remember…” line are verbatim; their order in the figure is not confirmed in fetched text — verify against Figure 2.)

Hyperparameters and setup. SFT: LR 5.5e-6 decaying (cosine) to 1.1e-6, batch 16, dropout 0.1, loss on target tokens only. DPO: LR 1e-6 decaying to 1e-7, batch 16, dropout 0.1, β **= 0.1**; checkpoints every 200 steps evaluated with Claude 2 on 253 validation examples for early stopping. Generation: N = 4, temperature 0.7, top-p 0.9; judge sampled 3 times.

Ablations that matter. Without EFT seed data the model produces far fewer valid pairs (541 and 429 vs 3,964 and 6,942) and lags more each iteration. Positive-only SFT does not improve over the baseline where DPO pairs do.

Code / data / license. No code or data release in fetched text. Venue ICML 2024.

Stated limitations. Only three iterations in one setting, saturation unknown; mean response length grows 1,092 → 1,552 → 2,552 tokens across M1–M3 and correlates with judged quality (length bias); reward hacking not ruled out; GPT-4 as evaluator and an LM as reward both need deeper analysis; no safety evaluation; smaller gains on math/reasoning; some NLP benchmarks flat or down.

Use in our design. For trajectories with no executable verifier (explanations, commit messages, design notes, reviews), the senior acts as judge with this rubric rewritten for code review (correctness, scope, tests, style), sampled 3× and averaged; pairs are highest-vs-lowest among N = 4 worker candidates, ties discarded, then DPO with $\beta = 0.1$ from the SFT adapter. Keep EFT-style judge examples (human-graded pairs with reasons) in the replay buffer so the judge does not drift. Monitor response length per iteration as the paper’s length drift is the known failure mode.

PDF: https://arxiv.org/pdf/2401.10020

2. Design extraction for the module

2.1 Algorithms to implement (consolidated pseudocode)

```
WEEKLY_LORA_JOB(domain d):
  base      <- frozen worker weights (NF4 + DQ, block 64/256)                    # QLoRA
  held_out  <- fixed held-out task set for d (never trained on) + transfer set (other domains)
  replay    <- ReplayBuffer[d]                                                    # all past weeks' kept
examples, tagged by week

  # ---- 1. Build this week's SFT set (STaR + ReST-EM) ----
  T_pass <- trajectories of the week with verifier.passed == true, grouped by task
  cap each task at 10 passing trajectories (ReST-EM)                             # avoid easy-task domination
  T_rat  <- for tasks the worker failed but a verified solution exists (senior or human):
              regenerate the worker's reasoning with the verified answer/patch as HINT (STaR rationalization)
              keep only if the regenerated trajectory passes the verifier; strip the hint before storing
  T_dist <- senior samples (several per task), verifier-filtered (ReST-EM Fig. 6 sequence-level distillation)
  SFT_week <- T_pass ∪ T_rat ∪ T_dist, loss masked to the completion (reasoning + actions), prompt excluded

  # ---- 2. Build pairs (DPO) ----
  P_ver   <- { (x, y_w = passing traj, y_l = failing traj) for same task x }
  P_human <- { (x, y_w = human-corrected, y_l = original) }
  P_judge <- for tasks without an executable verifier: N = 4 worker candidates, senior judge rubric, 3 samples
averaged,
             pair = (argmax score, argmin score), discard ties                   # Self-Rewarding
  DPO_week <- P_ver ∪ P_human ∪ P_judge

  # ---- 3. Thin week? synthesize tasks (Absolute Zero task factory, offline) ----
  if |SFT_week| < min_sft:
      propose (p, i) deduction/abduction tasks and induction tasks from repo code with K in-context references
      validate: runs without error, forbidden modules blocked, deterministic over j = 2 runs
      worker solves each G times; keep tasks with 0 < mean_solve < 1 (learnability band)
      passing solves -> SFT_week; pass/fail -> DPO_week

  # ---- 4. Train FROM BASE each week (STaR / ReST-EM) ----
  A_sft <- new LoRA(r, alpha, all linear layers, dropout) on base                # QLoRA defaults, 2.4
  train A_sft on SFT_week ∪ sample(replay, ratio)  with early stopping on held_out reward (ReST-EM Alg. 1)
  A_dpo <- copy(A_sft); pi_ref = base + A_sft (adapter frozen copy, or adapter disabled for ref logprobs)
  train A_dpo with L_DPO (beta = 0.1) on DPO_week ∪ sample(replay_pairs, ratio); eval every 200 steps (Self-
Rewarding)

  # ---- 5. Score, archive, gate ----
  E_new <- EVAL(A_dpo, held_out, transfer_set)                                   # 2.5
  archive(A_dpo, manifest{data counts, hyper, E_new, base hash, week})
  if E_new.heldOutPass1 >= E_active.heldOutPass1 + margin
     and E_new.transferPass1 >= E_active.transferPass1 - tolerance
     and E_new.meanLength <= E_active.meanLength * length_cap:
        activate(A_dpo)                                                          # merged W = W0 + BA for
serving, or hot-swap
  replay <- replay ∪ SFT_week ∪ DPO_week (tagged week)
```

Per-step details: - SFT update = ReST-EM Eq. (3) with $r \in \{0,1\}$: plain next-token loss on reward-1 samples. STaR schedule: start with a small number of steps and grow 20% per week if data grows. - DPO update = Eq. (7) in 1.3; implement the sequence log-prob as the sum over completion tokens for both π_θ and π_{ref}; the per-pair gradient weight is $\sigma\left(\hat{r}_\theta\left(x, y_l\right) - \hat{r}_\theta\left(x, y_w\right)\right)$ — log it as a diagnostic (pairs already ordered correctly contribute little). - Rationalization hint formats (STaR): put the known answer in the prompt in a fixed slot (“Target:” / “(CORRECT)” style) and remove it in the stored example. - Token-level KD (Hinton) only when senior and worker share a vocabulary: loss $\alpha T^2 \mathrm{CE}\left(p^{(T)}, q^{(T)}\right) + (1-\alpha)\,\mathrm{CE}\left(y, q^{(1)}\right)$, $T \in [2,4]$, $1-\alpha \leq 0.5$.

2.2 Equations and quantities the implementation must compute

1. Filtered-SFT objective (ReST-EM Eq. 3 / STaR gradient): $J(\theta) = \mathbb{E}_{(\mathbf{x},\mathbf{y}) \sim \mathcal{D}_i}\left[r(\mathbf{x},\mathbf{y})\, \log p_\theta(\mathbf{y} \mid \mathbf{x})\right]$ with $r \in \{0,1\}$ from the verifier.
2. DPO loss and implicit reward (1.3 Eq. 7 and $\hat{r}_\theta$), with $\beta = 0.1$; the completion log-prob $\log \pi(y \mid x) = \sum_t \log \pi\left(y_t \mid x, y_{<t}\right)$ over completion tokens only.
3. LoRA forward $h = W_0 x + \frac{\alpha}{r} BAx$, $A \sim$ Gaussian, $B = 0$ init; parameter count $|\Theta| = \sum_m r(d_m + k_m)$ over adapted matrices (reduces to $2\widehat{Ld}_{\text{model}} r$ for square ones); merge $W = W_0 + \frac{\alpha}{r} BA$.
4. QLoRA memory: quantized-matrix bits per parameter $4 + 0.127 = 4.127$ (NF4 + DQ, blocks 64/256) so a 9B parameter model’s quantized matrices take about $9 \times 10^9 \times 4.127/8 \approx 4.6$ GB (derived from the paper’s bit accounting; embeddings/lm_head kept in 16-bit add their own size). LoRA trainable parameters at r over all

linear layers plus optimizer state are the second term; activations under gradient checkpointing scale with sequence length and are the first thing to cut. QLoRA's own measurements: 7B 6.9 GB, 13B 11.3 GB total.

5. Learnability reward (AZR Eq. 4): $r_{\text{propose}} = 0$ if $\bar{r}_{\text{solve}} = 0$ else $1 - \bar{r}_{\text{solve}}$, $\bar{r}_{\text{solve}}$ the mean of G worker rollouts; used offline as a task filter (keep $0 < \bar{r}_{\text{solve}} < 1$).
6. Judge reward: mean of 3 sampled scores in $\{0..5\}$; pair = (max, min) if max ≠ min.
7. Distillation loss with temperature and the T^2 factor (1.7).
8. Early-stopping signal: held-out reward (pass@1) after each evaluation interval; stop when it stops improving (ReST-EM Alg. 1 "while reward improves on D_val").
9. Overfit alarm: training-set pass rate rising while held-out pass rate flat (ReST-EM Section 5.1).

2.3 Data structures and file formats (TypeScript-style type sketches)

```
        type Domain = string;                       // one LoRA pocket per domain, e.g. "ts-nextjs", "python",
"shell"
        type Week = string;                         // ISO week, e.g. "2026-W45"

        interface VerifierResult {
          passed: boolean;                          // the binary reward r in {0,1}
          checks: { name: string; passed: boolean; detail?: string }[];   // tests, lint, typecheck, executor
          source: "executor" | "tests" | "human" | "judge";
        }

        interface Trajectory {                      // one worker/senior/human run on one task
          id: string; taskId: string; domain: Domain; week: Week;
          prompt: string;                           // task + context as the worker saw it (few-shot included if
used)
          reasoning: string;                        // the reasoning trace (STaR rationale)
          actions: string;                          // patch / tool calls / final answer as text
          producer: "worker" | "senior" | "human";
          hintUsed: boolean;                        // true if produced by rationalization; hint must NOT be in
prompt
          verifier: VerifierResult;
          meta: { temperature: number; tokens: number; adapterId?: string };
        }

        interface SFTExample {                      // JSONL, one per line
          domain: Domain; week: Week;
          prompt: string; completion: string;       // completion = reasoning + actions; loss only on completion
          origin: "verified" | "rationalized" | "senior-distilled" | "synthetic-azr";
          taskId: string;                           // for the per-task cap (10)
        }

        interface PreferencePair {                  // JSONL, one per line
          domain: Domain; week: Week; taskId: string;
          prompt: string; chosen: string; rejected: string;
          source: "verified-vs-failed" | "human-corrected" | "judge";
          chosenScore?: number; rejectedScore?: number;   // judge means over 3 samples
        }

        interface SyntheticTask {                   // Absolute Zero triplet
          mode: "deduction" | "abduction" | "induction";
          program: string; input: string; output: string;      // o = p(i), executor-computed
          inductionIO?: { shown: [string, string][]; heldOut: [string, string][]; message: string };
          validity: { runsClean: boolean; safe: boolean; deterministic: boolean };   // j = 2 runs
          solveRate: number;                        // mean of G worker rollouts; keep if 0 < solveRate < 1
        }

        interface JudgeRecord {                     // EFT-style; also kept in replay so the judge does not
drift
          prompt: string; response: string; rubricVersion: string;
          justification: string; score: number;     // 0..5
          grader: "senior" | "human";
        }

        interface TrainHyper {
          rank: number; alpha: number; dropout: number; targetModules: "all-linear" | string[];
          quant: { type: "nf4"; doubleQuant: true; blockWeights: 64; blockConstants: 256; computeDtype: "bf16"
};
          sft: { lr: number; steps: number; batch: number; gradAccum: number; maxSeqLen: number; schedule:
"constant" | "linear"; warmup: number; maxGradNorm: number };
          dpo: { lr: number; beta: number; steps: number; batch: number; evalEvery: number };
          kd?: { temperature: number; softWeight: number };
          replayRatio: number;                      // fraction of each batch drawn from previous weeks
        }

        interface EvalResult {
          heldOutPass1: number; heldOutN: number;   // domain held-out set, verifier pass@1
          transferPass1: number;                    // other-domain / general set (ReST-EM Fig. 7 regression
check)
          trainPass1: number;                       // for the overfit alarm
          judgePairwiseAcc?: number;                // judge vs human-graded pairs (Self-Rewarding)
          meanCompletionTokens: number;             // length drift
          dpoRewardAccuracy?: number;               // fraction of held-out pairs with r_hat(y_w) > r_hat(y_l)
        }

        interface LoraPocket {                      // manifest.json next to adapter.safetensors
          id: string; domain: Domain; week: Week;
```

```
          baseModel: { name: string; sha256: string; quant: "nf4-dq" };
          trainedFrom: "base";                    // always base (STaR / ReST-EM); never a previous pocket
          hyper: TrainHyper;
          data: { sft: number; rationalized: number; distilled: number; synthetic: number; pairs: number;
replay: number };
          eval: EvalResult; previousActive?: string;
          status: "candidate" | "active" | "archived" | "rejected";
          files: { adapter: string; merged?: string };   // safetensors; merged GGUF/weights optional for
serving
        }

        interface ReplayBuffer { domain: Domain; sft: SFTExample[]; pairs: PreferencePair[]; judge:
JudgeRecord[]; }
```

File layout: `data/<domain>/<week>/{sft,pairs,synthetic,judge}.jsonl`; `pockets/<domain>/<id>/{adapter.safetensors,manifest.json}`; `pockets/<domain>/ACTIVE` symlink; `eval/<domain>/{heldout,transfer}.jsonl` (frozen; rotate only by explicit policy).

2.4 Hyperparameters and defaults to start from (with the paper each comes from)

Quantity	Default	Source
Base quantization	NF4 + double quantization, blocks 64 / 256, BF16 compute, paged optimizer, gradient checkpointing	QLoRA §3, App. B
LoRA target modules	all linear layers of every block	QLoRA Fig. 2 (“required to match full finetuning”)
LoRA rank r	16 (search 8–64; QLoRA: r unrelated to performance when all layers adapted; LoRA: r=1–8 enough for W_q, W_v)	QLoRA A.1, LoRA Tab. 6
LoRA alpha	16 (fix alpha, tune LR)	QLoRA B.2; LoRA §4.1
LoRA dropout	0.1	QLoRA B.2 (≤13B)
SFT LR (LoRA)	2e-4, constant	QLoRA Tab. 9 (7B/13B) ; LoRA GPT-2/GPT-3 2e-4
Max grad norm	0.3	QLoRA B.2
Adam beta2	0.999	QLoRA B.2
SFT batch	16 effective (batch 1 × grad-accum 16 in 12 GB)	QLoRA Tab. 9
SFT steps	start small, +20 % per week as data grows; stop on held-out plateau	STaR §3.1; ReST-EM Alg. 1
Per-task cap	10 passing trajectories	ReST-EM §5
Data-gen decoding	greedy / low temperature for the worker’s own positives; 0.7 (top-k 40) when sampling many candidates	STaR; ReST-EM
Samples per task for rejection sampling	32 (hard) … 64	ReST-EM
Iterations on the same task pool	≤ 3; stop when test gain flattens	ReST-EM §5.1
DPO beta	0.1 (0.5 if pairs are long-form and drift is a problem)	DPO App. B; Self-Rewarding
DPO LR	full-FT papers: 1e-6 (RMSprop, 150 warmup, batch 64) / 1e-6→1e-7 (batch 16); LoRA-DPO LR: not in these papers — start at the SFT-LoRA LR scaled down 10× and tune	DPO App. B; Self-Rewarding
DPO eval interval	every 200 steps (Self-Rewarding) / every 100 steps (DPO)	both
Candidates for judge pairs	N = 4, temperature 0.7, top-p 0.9; judge sampled 3×; drop ties	Self-Rewarding §2.2
KD temperature	2–4 (small student); soft term × T^2; hard weight ≤ 0.5	Hinton §2–3

Quantity	Default	Source
AZR validation	j = 2 determinism runs; forbidden modules os/sys/shutil…; S = 4 seed factor; format penalty −0.5 / −1	AZR §3.3
AZR learnability band	keep tasks with $0 < \bar{r}_{\text{solve}} < 1$	AZR Eq. 4
AZR RL LR (if ever used online)	AdamW 1e-6 constant, batch 64×6	AZR §4

2.5 Evaluation: how to know the module works (metrics, from the papers)

- **Held-out pass@1** on the domain's frozen held-out task set with the real verifier (ReST-EM's reward on $\mathcal{D}_{\text{val}}$; STaR's dev accuracy). A pocket is activated only if it beats the active pocket by a margin.
- **Transfer set**: pass@1 on tasks from other domains / a general coding set. ReST-EM saw HumanEval regress with APPS iterations; a drop beyond tolerance rejects the pocket even if the domain score rose.
- **Train-vs-held-out divergence** (ReST-EM §5.1): training pass rate rising while held-out is flat = overfitting; cut steps or iterations.
- **Saturation**: STaR stops when "performance saturates"; log the per-week held-out curve and stop re-training on an unchanged task pool after 2–3 passes.
- **Pass@k and majority vote** on the held-out set (ReST-EM Fig. 5, Table: 48.82 vs 44.02 maj@64) — a pocket that improves pass@1 but lowers pass@k has collapsed diversity.
- **DPO reward accuracy**: fraction of held-out pairs where $\hat{r}_\theta\left(x, y_w\right) > \hat{r}_\theta\left(x, y_l\right)$ (DPO Eq. 8 weighting; Self-Rewarding pairwise accuracy 65.1 → 81.7).
- **Judge quality**: senior/worker judge pairwise accuracy against human-graded pairs (Self-Rewarding Table: 5-best %, exact match, Spearman, Kendall).
- **Length drift**: mean completion tokens per pocket (Self-Rewarding 1,092 → 2,552 across iterations).
- **General-capability regression**: a small MMLU-style / general coding probe (QLoRA used MMLU 5-shot to show NF4 = 16-bit).
- **Memory**: peak VRAM per run must stay < 12 GB; QLoRA's 7B 6.9 GB / 13B 11.3 GB are the reference points.

2.6 Known pitfalls and failure modes (from the papers' limitations)

- **Bootstrapping needs above-chance ability** (STaR): a domain where the worker's few-shot pass rate is near zero yields no positives — use senior distillation / rationalization first.
- **Hint leakage** (STaR): if the hint stays in the training prompt the model learns to copy the hinted answer with unsound reasoning. Strip hints; verify rationalized traces still pass.
- **Unfaithful rationales** (STaR): answer-first-then-rationalize; prefer traces whose intermediate actions were executed (tool calls) over pure text reasoning.
- **High temperature produces right answers from wrong reasoning** (STaR): keep worker positives at low temperature.
- **Overfitting across iterations on small pools** (ReST-EM: APPS regressed at iteration 2; STaR: continuing from M_{n-1} overfits): train from base, cap iterations, cap per-task samples, early-stop on held-out.
- **Transfer regression** (ReST-EM Fig. 7): continuing from the previous checkpoint harms held-out transfer; train from base and check the transfer set.
- **Easy-task domination**: without the per-task cap, many trivial passes dominate the SFT set (ReST-EM cap 10).
- **DPO reference mismatch** (DPO §4): π_{ref} must be the distribution the pairs were sampled from; if pairs come from an older pocket, re-SFT the reference on the chosen completions first.
- **DPO over-optimization and length bias** (DPO §7; Self-Rewarding): monitor length and reward accuracy; raise β if drift appears.
- **Judge reward hacking** (Self-Rewarding): the judge and the policy are the same family; keep human-graded judge examples in replay and re-measure judge accuracy each week.
- **LoRA on attention only is insufficient for decoder LLMs** (QLoRA Fig. 2); rank matters little once all layers are covered.
- **QLoRA not validated against full 16-bit FT above 13B** (QLoRA §8) — irrelevant at 9B but note it for the senior.
- **Token-level KD needs a shared vocabulary** (Hinton's loss is defined over the same class set) — otherwise use sequence-level distillation.
- **Executor safety** (AZR "uh-oh moment", forbidden modules): synthetic tasks and worker solutions run only in the sandbox with the module blocklist and determinism check.
- **Batching different adapters in one forward pass is not straightforward** (LoRA §4.2): serve one active pocket per domain, merge for speed.

2.7 Open questions this section does not settle

- LoRA-specific learning rates for DPO: all DPO numbers here are full fine-tuning (1e-6); the right LoRA-DPO LR for a 9B NF4 worker must be found empirically.
- Exact AZR values for K (references), G (rollouts), λ, N (induction inputs) and the RL coefficients: not in fetched text — read the paper's appendix hyperparameter table before porting the task factory.
- Replay ratio and how many weeks to retain: none of the papers trains with an explicit replay buffer; ReST-EM/STaR instead retrain from base on the *whole* current pool. Whether "base + (all weeks' data)" or "base + (this week + sampled replay)" is better at our data scale is untested.
- Whether a merged pocket (W0 + BA quantized back to NF4/GGUF for llama.cpp serving) loses accuracy relative to the un-merged adapter on the BF16-dequantized path — not covered by these papers.
- Reference policy for DPO under QLoRA: computing π_{ref} by disabling the adapter (same base, no extra copy) is an implementation inference, not from the papers; verify it equals the SFT model used to sample the pairs.
- Rationalization hint format for free-form coding tasks (STaR only shows arithmetic/CQA hints).
- Judge rubric for code: the Self-Rewarding rubric is for assistant answers; a code-review rubric with the same additive 5-point structure needs its own EFT seed set and validation.
- Stopping rule across weeks (STaR "saturation") when the task distribution itself drifts with the user's work.

3. Verified source table

arXiv id	Title	Year	Venue	Code	PDF
2203.14465	STaR: Bootstrapping Reasoning With Reasoning	2022	NeurIPS 2022 (not on abs page)	https://github.com/ezelikman/STaR	https://arxiv.org/pdf/2203.14465
2312.06585	Beyond Human Data: Scaling Self-Training for Problem-Solving with Language Models (ReST-EM)	2023	TMLR 04/2024	none	https://arxiv.org/pdf/2312.06585
2305.18290	Direct Preference Optimization: Your Language Model is Secretly a Reward Model	2023	NeurIPS 2023 (not on abs page)	https://github.com/eric-mitchell/direct-preference-optimization	https://arxiv.org/pdf/2305.18290
2505.03335	Absolute Zero: Reinforced Self-play Reasoning with Zero Data	2025	arXiv preprint (v3 Oct 2025)	https://github.com/LeapLabTHU/Absolute-Zero-Reasoner	https://arxiv.org/pdf/2505.03335
2106.09685	LoRA: Low-Rank Adaptation of Large Language Models	2021	ICLR 2022 (not on abs page)	https://github.com/microsoft/LoRA	https://arxiv.org/pdf/2106.09685
2305.14314	QLoRA: Efficient Finetuning of Quantized LLMs	2023	NeurIPS 2023 ("Extended NeurIPS submission")	https://github.com/artidoro/qlora	https://arxiv.org/pdf/2305.14314

arXiv id	Title	Year	Venue	Code	PDF
1503.02531	Distilling the Knowledge in a Neural Network	2015	NIPS 2014 Deep Learning Workshop (not on abs page)	none	https://arxiv.org/pdf/1503.02531
2401.10020	Self-Rewarding Language Models	2024	ICML 2024	none	https://arxiv.org/pdf/2401.10020